

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Transactional Memory

Instructor: Prof. Joshua San Miguel

Some slides adopted from Greg Steffan and Natalie Enright Jerger

Last Lecture

- Directory-based coherence – MESI
- Coherence optimizations
- Virtual networks

This Lecture

- Optimistic concurrency
- Version management and conflict detection
- SW and HW transactional memory

Which is Better?

coarse-grained lock

```
parallel task {  
    lock(ALL);  
    if (A == ...) {  
        // do something with X  
        ...  
        // do something with Y  
        A = ...;  
    }  
    unlock(ALL):  
}
```

VS

fine-grained locks

```
parallel task {  
    lock(A);  
    if (A == ...) {  
        unlock(A);  
        lock(X);  
        // do something with X  
        unlock(X);  
        ...  
        lock(Y);  
        // do something with Y  
        unlock(Y);  
        lock(A);  
        A = ...;  
    }  
    unlock(A):  
}
```



Optimistic Concurrency

Synchronization paradigm that allows code regions to execute in parallel yet maintain the appearance of atomic, serial execution.

- Best of both worlds: coarse-grained vs. fine-grained locking
- All-or-nothing: each execution either *commits* all state changes or *rolls back* and tries again
- e.g., Speculative lock elision

Transactional Memory

Optimistic concurrency mechanism with transactional interface for reading from and writing to shared memory.

```
thread {  
  
    if (A == ...) {  
        // do something with X  
        ...  
        // do something with Y  
        A = ...;  
    }  
  
}
```

Transactional Memory

Optimistic concurrency mechanism with transactional interface for reading from and writing to shared memory.

```
thread {  
    atomic {  
        if (A == ...) {  
            // do something with X  
            ...  
            // do something with Y  
            A = ...;  
        }  
    }  
}
```

Transactional Memory – Registers

Begin Transaction:

Save register state

Execute Transaction:

Read and write registers freely

Commit Transaction:

- If no conflicts, free saved register state
- If conflicts, roll back, restore saved register state

Transactional Memory – Shared Memory

Begin Transaction:

Initialize internal structures (e.g., undo logs, write buffers)

Execute Transaction:

Track reads and writes to memory, with potential conflicts (i.e., atomicity violations)

Commit Transaction:

- If no conflicts, push any buffered writes to memory
- If conflicts, roll back, undo any in-place writes to memory

Transactional Memory

Version Management:

How do we track reads and writes to memory that may have potential conflicts?

- Eager vs. lazy

Conflict Detection:

How do we identify and resolve conflicts between multiple in-flight transactions?

- Eager vs. lazy

Transactional Memory

Read Set: List of shared memory addresses read in transaction

Write Set: List of shared memory addresses modified in transaction

- Constructed on-the-fly (i.e., addresses not known a priori)
- Can be any granularity (e.g., words/objects in SW, blocks in HW)
- False positives are okay (e.g., bloom filters [D. Sanchez et al., MICRO 2007])

Transactional Memory

Read Set: List of shared memory addresses read in transaction

Write Set: List of shared memory addresses modified in transaction

- Constructed on-the-fly (i.e., addresses not known a priori)
- Can be any granularity (e.g., words/objects in SW, blocks in HW)
- False positives are okay (e.g., bloom filters [D. Sanchez et al., MICRO 2007])

```
atomic {  
    if (accts[ID_FROM].balance >= AMT) {  
        accts[ID_FROM].balance -= AMT;  
        accts[ID_TO].balance += AMT;  
    }  
}
```

Transactional Memory

Read Set: List of shared memory addresses read in transaction

Write Set: List of shared memory addresses modified in transaction

- Constructed on-the-fly (i.e., addresses not known a priori)
- Can be any granularity (e.g., words/objects in SW, blocks in HW)
- False positives are okay (e.g., bloom filters [D. Sanchez et al., MICRO 2007])

```
atomic {
```

```
    if (accts[ID_FROM].balance >= AMT) {  
        accts[ID_FROM].balance -= AMT;  
        accts[ID_TO].balance += AMT;  
    }
```

```
}
```

read set = {accts[ID_FROM].balance, accts[ID_TO].balance}
write set = {accts[ID_FROM].balance, accts[ID_TO].balance}



Eager Version Management

Upon a store instruction:

Write value directly to memory

If address not in Write Set:

1. Buffer old value in Undo Log
2. Add address to Write Set

Upon a load instruction:

Read value directly from memory

Upon committing transaction:

Clear Undo Log and Write Set

Upon rolling back transaction:

Write values in Undo Log back to memory

Clear Undo Log and Write Set

```
atomic {
```

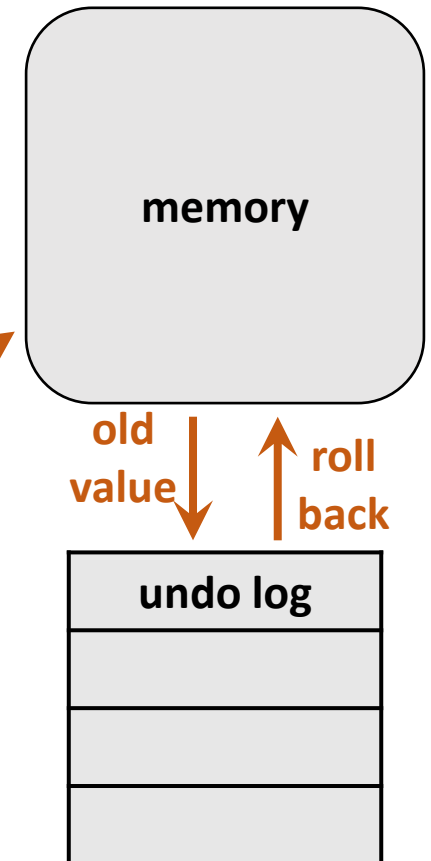
```
  *x = ...;
```

```
  *y = ...;
```

```
  ... = *z;
```

```
  ...
```

```
}
```



Lazy Version Management

Upon a store instruction:

Don't write value to memory yet; put value into Write Buffer/Set instead

Upon a load instruction:

If address in Write Set: read value from Write Buffer

Otherwise: read value from memory

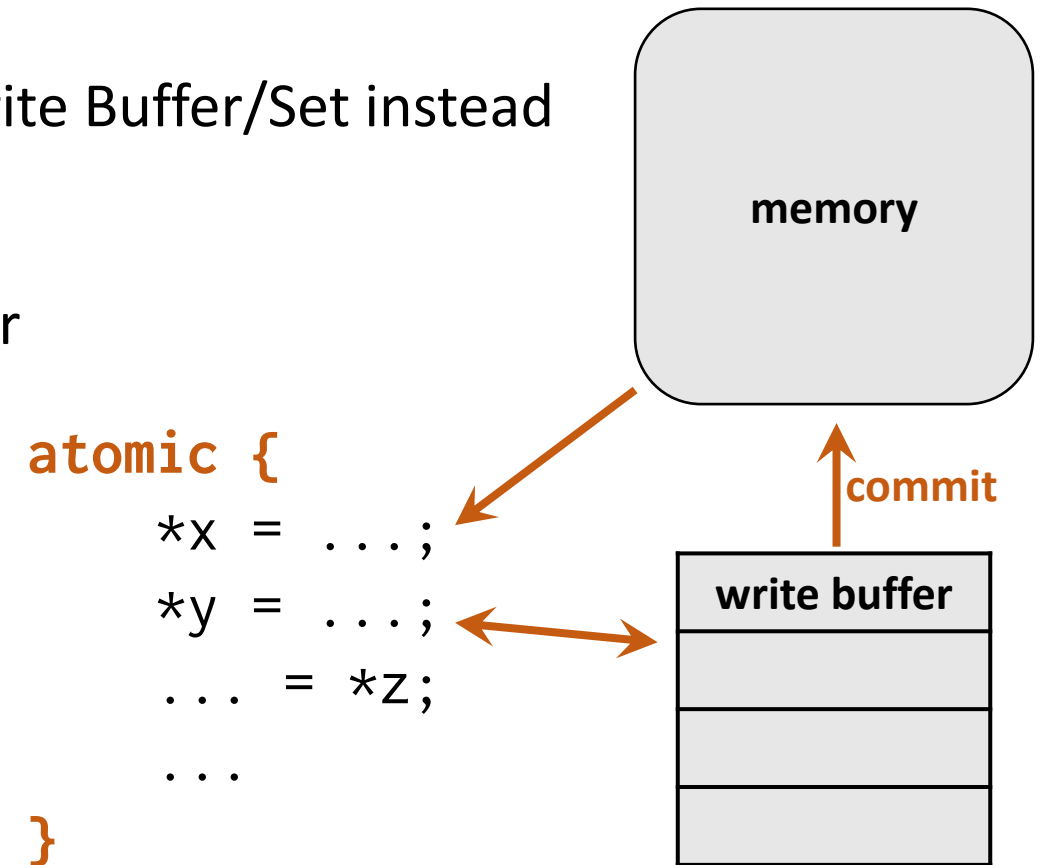
Upon committing transaction:

Write values in Write Buffer to memory

Clear Write Buffer and Write Set

Upon rolling back transaction:

Clear Write Buffer and Write Set



Eager vs. Lazy Version Management

When to use Eager

- Few store instructions (potentially two writes per store during txn)
- Low conflict rate (lots of commits with few rollbacks)

VS

When to use Lazy

- Few load instructions (potentially two reads per load during txn)
- High conflict rate (many rollbacks)

Eager vs. Lazy Version Management

When to use Eager

- Few store instructions (potentially two writes per store during txn)
- **Low conflict rate (lots of commits with few rollbacks)**

VS

When to use Lazy

- Few load instructions (potentially two reads per load during txn)
- High conflict rate (many rollbacks)

key motivation for optimistic concurrency
in the first place

Transactional Memory

Version Management:

How do we track reads and writes to memory that may have potential conflicts?

- Eager vs. lazy

Conflict Detection:

How do we identify and resolve conflicts between multiple in-flight transactions?

- Eager vs. lazy

Conflict Detection

Need to roll back the transaction(s) if a conflict has occurred or if a future conflict may go undetected

- **Eager:** Check for conflicts at every load and store
- **Lazy:** Check for conflicts at commit time

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**

my {	T {
*A = 1;	... = *A;
...	...

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**

my {	T {
*A = 1;	... = *A;
*B = 1;	
...	...

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back?**

<pre>my { *A = 1; *B = 1; }</pre>	<pre>T { ... = *A; ... = *B; }</pre>
---	---

Eager Conflict Detection – Eager VM

Upon committing transaction:

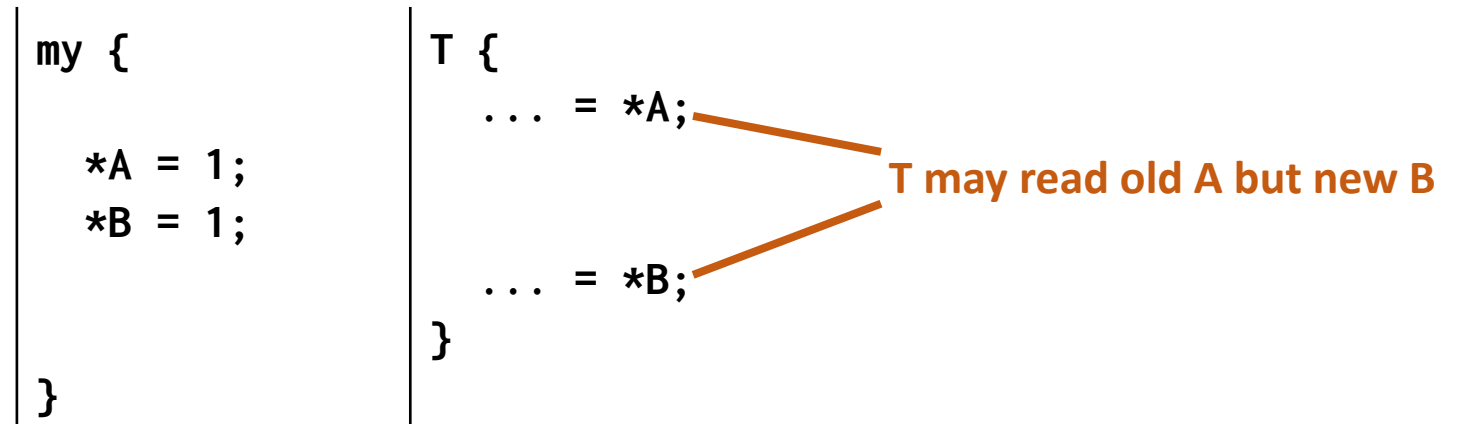
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**

balance between evicting transaction that
started later vs. is further from finish line

```
my {  
    *A = 1;  
    *B = 1;  
}
```

```
T {  
    ... = *A;  
    ... = *B;  
}
```

T may read old A but new B



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
  
    *A = 0;  
    ...
```

```
T {  
    *A = 1;  
  
    ...
```



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

memory may end up with
my A but T's B

```
my {  
    *A = 0;  
    *B = 0;  
}
```

```
T {  
    *A = 1;  
  
    *B = 1;  
}
```

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

memory may end up with
my A but T's B

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**

<pre>my { *A = 0; *B = 0; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
---	--

if rolling back T, don't undo
stale write to A



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction**

```
my {  
    *A = 0;  
}
```

```
T {  
    *A = 1;  
    *B = 1;  
  
    if(*A == *B)  
        ...  
}
```

may be false



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

<pre>my { ... = *A; ...</pre>	<pre>T { *A = 1; ...</pre>
--	--

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

my {	T {
	*A = 1;
... = *A;	
... = *B;	
...	...



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

<pre>my { ... = *A; ... = *B; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
---	--

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

<pre>my { ... = *A; ... = *B; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
---	---

not a problem;
T's store can detect conflict later



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

my {	T {
	*A = 1;
... = *A;	
... = *B;	
}	
	*B = 1;
	}

Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

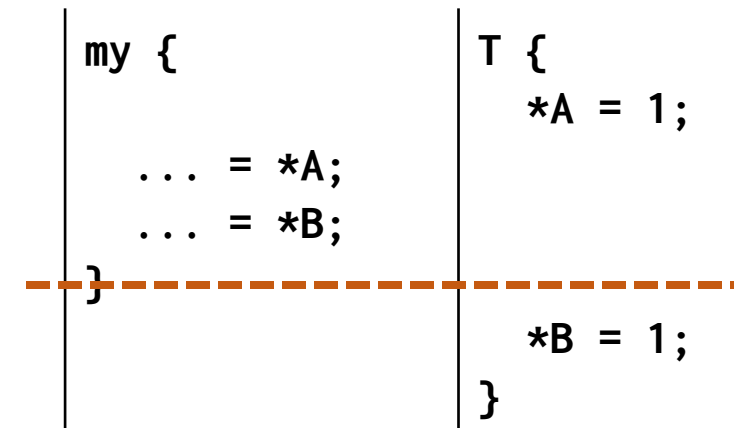
Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction**

may read new A but old B,
before T can detect it



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction**

may read new A but old B,
before T can detect it

<pre>my { ... = *A; ... = *B; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	--

if rolling back T, perform load after



Eager Conflict Detection – Eager VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: roll back either transaction

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: roll back either transaction

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

my {	T {
*A = 1;	... = *A;
*B = 1;	... = *B;
}	}

Eager Conflict Detection – Lazy VM

Upon committing transaction:

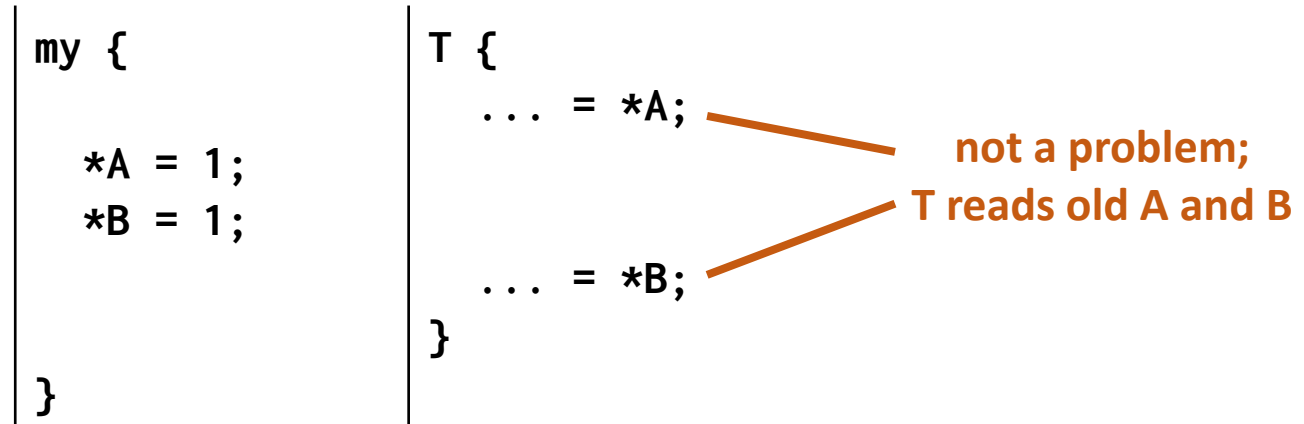
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction?**

```
my {  
    *A = 1;  
    *B = 1;  
}
```

```
T {  
    ... = *A;  
  
    ... = *B;  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

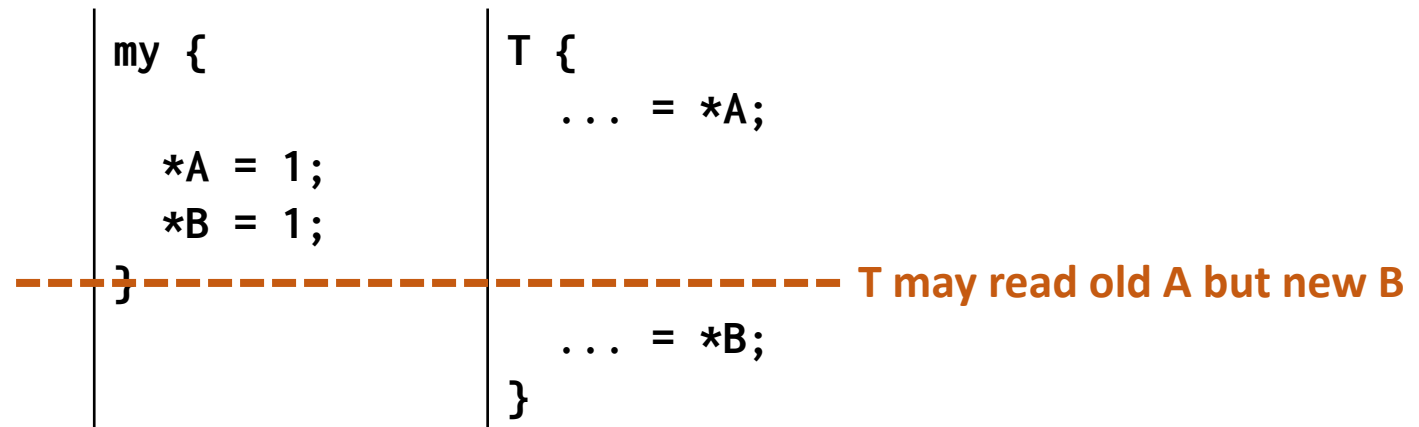
Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

➤ If X is in T's Read Set: **roll back either transaction**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
    *A = 0;  
    *B = 0;  
}
```

```
T {  
    *A = 1;  
  
    *B = 1;  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

<pre>my { *A = 0; *B = 0; }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
---	---

no problem; overwrites A



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **roll back either transaction?**

```
my {  
  
    *A = 0;  
}
```

```
T {  
    *A = 1;  
    *B = 1;  
  
    if(*A == *B)  
        ...  
}
```



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: **do nothing**

<pre>my { *A = 0; }</pre>	<pre>T { *A = 1; *B = 1; if(*A == *B) ... }</pre>
--	--

no problem;
reads from private write buffer



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction?**

<pre>my { if(*A == *B) ... }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	---

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

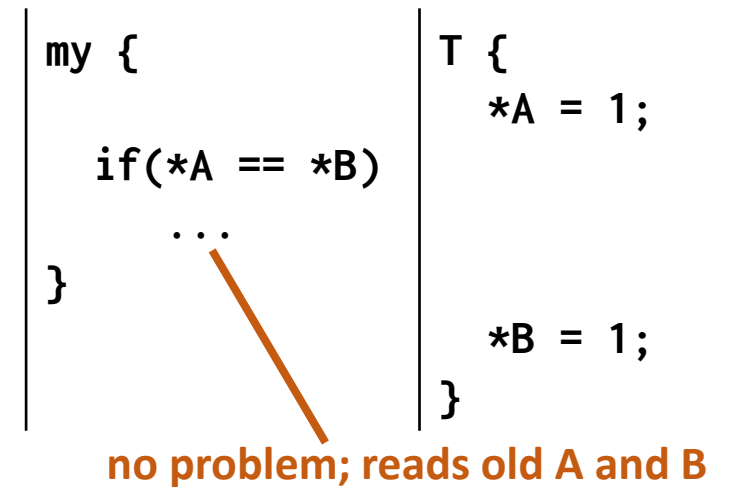
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**

<pre>my { if(*A... ...== *B) ... }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	--

Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

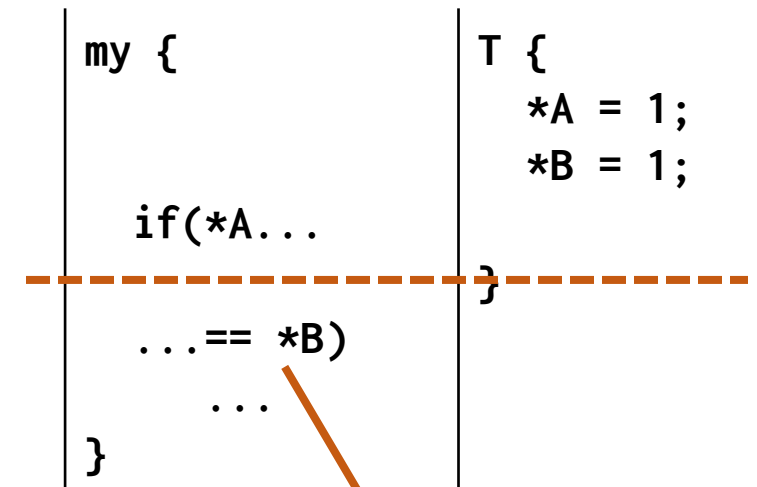
- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **do nothing?**



may read old A but new B



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: **roll back either transaction**

<pre>my { if(*A... ...== *B) ... }</pre>	<pre>T { *A = 1; *B = 1; }</pre>
--	--



Eager Conflict Detection – Lazy VM

Upon committing transaction:

Clear Write Set and Read Set

Upon a store instruction:

Add address X to Write Set (if not already there)

For each in-flight transaction T:

- If X is in T's Read Set: roll back either transaction
- If X is in T's Write Set: do nothing

Upon a load instruction:

Add address X to Read Set (if not already there)

For each in-flight transaction T:

- If X is in T's Write Set: roll back either transaction